Introduction

Introduction I

This `template_eda_notebook` serves as the exploratory-data-analysis exemplar for the Research Project Template ecosystem, demonstrating how a computational notebook can stay reproducible by delegating every computation to a fully-tested library. The prose, the labelled figures, and the summary table are all produced through an auditable custody chain: a deterministic dataset, tested functions in `src/eda/`, a thin analysis script, and multi-format rendering.

Why exploratory data analysis I

EDA — the work of getting to know a dataset before committing to a model — is where most research projects actually begin: load the data, see how much is missing, look at distributions, and check which variables move together. It is fast and interactive by nature, which is exactly why it tends to live in notebooks. The hazard is that the interactive convenience of a notebook cell is also a trap: a one-off `df.groupby(...).mean()` becomes load-bearing, never gets a test, and silently disagrees with the figure two cells down after the data changes.

The notebook -> tested src extraction workflow I

This exemplar teaches one discipline: **explore in a cell, then extract to a tested library the moment a computation matters.** Concretely:

The notebook -> tested src extraction workflow I

1. The walkthrough notebook (`notebooks/eda_walkthrough.ipynb`) imports from `src` and calls tested functions; it contains no business logic of its own.
2. Each analytical step — loading, cleaning, summarizing, correlating, preparing figure data — is a typed, documented function in `src/eda/` with a test that asserts an exact numeric property.
3. A thin script (`scripts/eda_analysis.py`) runs the same pipeline headless and writes the figures and a summary CSV to `output/`.

Template architecture context I

The project sits on the repository's three pillars:

Template architecture context I

1. `src/eda/` **library**: pure pandas/numpy data transforms — no plotting, no file I/O, no `infrastructure` imports. This purity is what makes the library forkable and trivially testable.
2. `tests/` **framework**: a zero-mock suite that exercises the library against the shipped CSV and tiny real frames, plus a structural check that the notebook's imports bind to the library's public surface.
3. `docs/` **knowledge base**: architectural guidelines, the testing philosophy, and the operational rules that govern agents editing this tree.

The dataset I

We analyze a small synthetic cohort of subject measurements — height (cm), weight (kg), and resting heart rate (bpm) across three groups. The dataset is a static, committed fixture — the same file on every run — so every statistic in sec. `sec:results` is reproducible. The data is shaped so that weight depends positively on height (a strong, easy-to-see correlation) while resting heart rate is only weakly related, and a few cells are left blank to exercise the missing-data path honestly.

Reader's guide to the manuscript I

Reader's guide to the manuscript I

- **sec.** `sec:methodology` ties each EDA step to its function in `src/eda/`.
- **sec.** `sec:results` is figure-centric: each panel names the figure-data preparer that produced it.
- **sec.** `sec:experimental_setup` lists the dataset schema and software environment.
- **sec.** `sec:reproducibility` records the artifact inventory and the exact commands to regenerate everything.
- **sec.** `sec:scope` states scope and related literature so the exemplar is not mistaken for a general-purpose EDA toolkit.